SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I SARAVANAN.T (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SARAVANAN.T

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ASSESSMENT TOOL 2 — DETAILED ANSWERS

Course: Artificial Intelligence | Course Code: CSA17

Topic: Constraint-Based Problem Solving

The original question paper is included first. Detailed answers are added after it in the same PDF.

ANSWER 1: Backtracking Search for Doctor Shift Assignment

Problem statement

A hospital must assign doctors D1, D2 and D3 to Morning, Afternoon and Night shifts. Each doctor must receive exactly one shift, and each shift must contain exactly one doctor.

Variables and domains

Let D1, D2 and D3 represent the shifts assigned to the three doctors. The domain for each variable is {Morning, Afternoon, Night}.

Constraints

• D1 cannot work the Night shift.

• D2 must work before D3; the shift order is Morning < Afternoon < Night.

• Only one doctor can be assigned to each shift.

• D3 cannot work the Morning shift.

• Every doctor must be assigned exactly one shift.

Backtracking search: step-by-step

Step 1: Assign D2 to Morning. This is a tentative assignment. D3 can still be assigned Afternoon or Night, so no constraint is violated.

Step 2: Try assigning D3 to Afternoon. D2 is before D3, and D3 is not in Morning. This partial assignment is valid.

Step 3: The remaining shift is Night, so try assigning D1 to Night. This violates the rule that D1 cannot work Night. Reject this assignment and backtrack.

Step 4: Change the assignment by placing D3 in Night. D2 remains in Morning, so D2 is before D3. Assign D1 to the remaining Afternoon shift. All constraints are now satisfied.

Constraint checking

Constraint	Final assignment	Result
D1 cannot work Night	D1 = Afternoon	Satisfied
D2 must work before D3	D2 = Morning; D3 = Night	Satisfied
One doctor per shift	Each shift has one doctor	Satisfied
D3 cannot work Morning	D3 = Night	Satisfied
Each doctor gets one shift	D1, D2 and D3 each assigned once	Satisfied

Final valid schedule

Shift	Doctor
Morning	D2
Afternoon	D1
Night	D3

Conclusion: Backtracking builds a solution one assignment at a time. When D1 is assigned to Night, the algorithm detects a violation and returns to try a different assignment. The final schedule satisfies all five constraints.

ANSWER 2: Robot Navigation Using BFS

Problem statement

A robot must move from Start (S) to Goal (G) in a 5 × 5 grid. It can move Up, Down, Left or Right. Each move costs 1, obstacles cannot be crossed, and the question mentions Manhattan Distance as a heuristic.

Breadth-First Search (BFS)

BFS is an uninformed search algorithm that explores nodes level by level. It uses a FIFO (First In, First Out) queue: the node added earliest is removed first. When every move has the same cost, BFS finds a path with the minimum number of moves, if the goal is reachable.

Algorithm and node expansion

Step 1 — Initialize: Insert S into the queue, mark S as visited and set its path cost to 0. Queue: [S].

Step 2 — Expand: Remove the first node from the queue. Examine its neighbours in the permitted directions: Up, Down, Left and Right.

Step 3 — Check neighbours: Reject any cell outside the grid, any obstacle, and any cell already visited. For every valid unvisited cell, record its parent and cost, mark it visited, and add it to the back of the queue.

Step 4 — Repeat: Continue removing and expanding nodes in FIFO order. Each newly reached cell has a path cost one greater than its parent.

Step 5 — Goal test: Stop when G is reached. Follow the recorded parent links backward from G to S to reconstruct the path.

Queue update template

Iteration	Node removed / expanded	Valid unvisited neighbours added	Queue after update
1	S	All valid adjacent cells	Remaining queue followed by neighbours
2	First node in queue	Its valid unvisited neighbours	Remaining queue followed by new neighbours
3	Next node in queue	Its valid unvisited neighbours	Remaining queue followed by new neighbours
…	Continue in FIFO order	Ignore obstacles and visited cells	…
Final	G reached	Stop and reconstruct path	Search ends

Manhattan Distance

The Manhattan Distance heuristic estimates the number of horizontal and vertical moves between a cell and the goal:

$h(n) = |x_n - x_G| + |y_n - y_G|$

Here, (x_n, y_n) is the current cell and (x_G, y_G) is the goal cell. It does not account for detours caused by obstacles.

Optimal path and cost

Because the supplied PDF does not show the actual grid or the locations of S, G and the obstacles, the exact node-expansion sequence, path and total cost cannot be calculated from the provided information. They depend on the missing grid.

When the grid is known, calculate the path cost as: **Total cost = number of moves × 1**. BFS finds a shortest path by number of moves because all moves have equal cost.

Important distinction: Standard BFS does not use Manhattan Distance to prioritize the queue. If nodes are prioritized using a heuristic, that is a different strategy, such as A* search. The question's grid is needed to give the exact path without inventing obstacle positions.

Conclusion: BFS uses a FIFO queue, expands cells level by level, avoids blocked and visited cells, and returns a shortest path when a reachable goal exists and every movement has equal cost.

ANSWER 3: Autonomous Rescue Robot

(a) Analyze the constraints and their effect on decision-making

The robot must travel from S to a survivor at G in a disaster-affected building. Because it can observe only adjacent cells, it must make decisions with incomplete information and update its internal map as it explores.

• **Movement:** It can move Up, Down, Left or Right only, so diagonal moves are not allowed.

• **Obstacles:** Blocked cells cannot be entered. When one is detected, the robot records it and searches for another route.

• **Limited sensing:** The robot observes adjacent cells rather than the entire building. Unknown cells must not be treated as confirmed safe or blocked until information is obtained.

• **Movement cost:** Each movement costs 1.

• **Risk cost:** Entering a risky zone adds 2, so the total cost of entering such a zone is 1 + 2 = 3.

• **Cost minimization:** The robot compares complete route costs, not just the number of steps.

• **Dynamic knowledge:** Newly discovered obstacles or risks may change the best route, so the robot updates its map and replans.

Conclusion: The constraints require the robot to avoid known obstacles, account for risk, learn about unknown cells and continuously choose a feasible low-cost route.

(b) Solution using Uniform Cost Search and online exploration

Uniform Cost Search (UCS) expands the candidate with the smallest accumulated path cost. Online exploration allows the robot to gather information and update its map while operating in the initially unknown building.

Step 1: Start at S and set g(S) = 0. Create an internal map and mark unexplored cells as unknown.

Step 2: Sense adjacent cells and classify them as free, blocked, risky or unknown.

Step 3: Assign cost 1 to entering a normal cell and cost 3 to entering a risky cell.

Step 4: Maintain candidate routes in a priority queue ordered by accumulated cost. Expand the candidate with the lowest cost among the known options.

Step 5: Move or explore, record newly observed information, and update the map. Do not traverse known blocked cells.

Step 6: If new information makes a planned route invalid or more costly, update the available routes and replan.

Step 7: Continue until G is reached, then calculate the total cost of the selected route.

Formula: **Route cost = sum of movement costs + sum of additional risk costs.**

Example calculation only: If a route has 6 movements and enters 2 risky zones, the movement cost is 6 × 1 = 6 and the extra risk cost is 2 × 2 = 4. Total = 6 + 4 = 10. This is an example, not the actual cost of the unspecified grid.

Conclusion: UCS compares accumulated path costs; online exploration allows the robot to update its knowledge and replan when it discovers new obstacles or risky cells.

(c) Application of AI concepts

1. Intelligent agent: The robot is an intelligent agent because it senses the environment, stores information, chooses actions and moves toward a goal. Sensors provide observations, an internal map stores knowledge, the search algorithm selects an action, and actuators perform movement.

2. Rationality: A rational robot chooses an action that best achieves its goal based on the information available. It considers whether a cell is blocked, whether it is risky, and the accumulated cost of the route. For example, a route costing 8 is preferable to a route costing 10 if both are feasible and satisfy the same safety requirements.

3. Environment type:

• **Partially observable:** only adjacent cells can be sensed.

• **Sequential:** each move affects later choices.

• **Partially known:** the complete map is not available initially.

• **Cost-based:** normal and risky cells have different costs.

• **Uncertain information:** obstacles and risks may be unknown before exploration.

• **Dynamic knowledge:** the robot's map changes as new information is observed; the physical building itself need not change.

Conclusion: The robot demonstrates intelligent-agent behaviour through sensing, reasoning and action. Its rationality is judged by how well it achieves the rescue goal while managing route cost and safety.

(d) Justification of the chosen approach

• **Different costs:** UCS can compare normal and risky movements because it uses accumulated path cost.

• **Least-cost solution:** With non-negative costs and a correctly represented search space, UCS finds a minimum-cost route to a reachable goal.

• **Obstacle avoidance:** Known blocked cells can be excluded from valid transitions.

• **Online adaptation:** The robot can gather information as it explores, update its map and reconsider routes.

• **Rational choices:** It can choose a route with a greater number of movements if its total cost is lower.

A limitation is that ordinary UCS assumes the transition costs being evaluated are known. In a partially observable environment, the robot must explore and replan as information becomes available. Minimum numerical cost alone does not guarantee absolute physical safety unless safety requirements are explicitly represented.

Final conclusion: UCS combined with online exploration is suitable for this rescue problem because it supports cost minimization, obstacle avoidance and decisions under partial observability. The robot senses nearby cells, updates its internal map and adapts its route to help reach the survivor safely.